REPORTE

Análisis Léxico en Java — Patrones, Lexemas y Tokens

ALUMNO: Ángel Ortega Gómez
Dr. Eduardo Cornejo-Velázquez

1. Decisiones de diseño

1.1 Caso 1

El único cambio que realicé fue en el método toString() del token. El código original utilizaba el símbolo de flecha (→) para mostrar la salida, pero en mi entorno, este carácter no se mostraba correctamente y aparecía como un signo de interrogación (?).

Para evitar este problema, decidí reemplazarlo por una versión más simple usando caracteres normales (–>). De esta manera, la salida se muestra correctamente sin depender de configuraciones especiales de codificación.

Este cambio no afecta el funcionamiento del analizador, pero sí mejora la claridad de la salida y evita errores visuales.

1.2 Caso 2

En este caso, el código presentado en la práctica amplía el analizador para reconocer operadores y separadores, agregando nuevas expresiones regulares dentro del patrón MASTER, así como nuevas condiciones dentro del proceso de tokenización.

Para poder integrar correctamente estos nuevos elementos, fue necesario agregar los nuevos tipos de token (OPERATOR y SEPARATOR) dentro del enum TokenType, ya que en la versión anterior solo se contemplaban palabras reservadas, identificadores y espacios en blanco.

Además, el patrón de operadores se separó en una constante independiente, de forma similar a como se manejan las palabras reservadas (KW_PATTERN). Esto se hizo principalmente por claridad y organización del código, ya que la expresión regular de operadores es extensa y resulta más fácil de leer y mantener al estar definida por separado.

En general, la implementación se realizó siguiendo la lógica indicada, integrando los nuevos patrones y asegurando que el analizador pudiera clasificar correctamente estos símbolos.

1.3 Caso 3

En este caso, el código presentado en la práctica amplía el analizador para reconocer distintos tipos de literales, incluyendo números en diferentes formatos (enteros, flotantes, binarios, octales y hexadecimales), así como cadenas (String) y caracteres (char).

Al igual que en el caso anterior, fue necesario agregar todos estos nuevos tipos dentro del enum TokenType, para que el analizador pudiera clasificarlos correctamente.

Además de integrar las expresiones regulares y las condiciones dentro del while, se realizó un ajuste importante en el orden de los patrones dentro del MASTER. En la versión final, los patrones de STRINGLIT y CHARLIT se colocaron antes que los literales numéricos.

Este cambio se hizo porque el orden en el que se evalúan las expresiones regulares influye directamente en el resultado del análisis. Al colocar primero las cadenas y caracteres, se evita que partes de estos puedan ser interpretadas incorrectamente como otros tipos de tokens, lo que mejora la precisión del analizador.

En general, estos ajustes permitieron que el programa reconociera correctamente una mayor variedad de elementos del lenguaje, haciéndolo más completo y más cercano a un analizador léxico real.

1.4 Caso 4

En este caso, el código presentado en la práctica introduce el manejo de errores léxicos mediante una clase específica (LexicalError), así como una mejora en la organización del patrón MASTER.

Para poder integrar correctamente esta funcionalidad, se realizaron algunos ajustes. Primero, la clase de error se adaptó para que pudiera utilizarse dentro del programa, por lo que se definió de forma estática, permitiendo registrar información como línea, columna y carácter inválido.

También fue necesario declarar una lista de errores (`List<LexicalError> errors = new ArrayList<>();`) dentro del método de tokenización, ya que esta no estaba incluida originalmente y es necesaria para almacenar los errores detectados durante el análisis.

Además, se agregaron los nuevos tipos correspondientes en el enum TokenType para contemplar los comentarios (JAVADOC, BLOCKCOMMENT y LINECOMMENT). Sin embargo, aunque estos tipos existen, no se agregaron condiciones dentro del while para incluirlos en la lista de tokens, ya que en este caso se indica que los comentarios no deben generar salida, sino únicamente ser reconocidos por el analizador.

Otro punto importante fue la implementación de la estrategia conocida como panic recovery, donde, si el analizador encuentra texto que no coincide con ningún patrón, avanza un carácter, registra el error y continúa con el proceso. Esto permite que el programa no se detenga ante el primer error, sino que siga analizando el resto del archivo.

Por otro lado, se reorganizó el orden de las expresiones dentro del patrón MASTER. Se colocaron primero los comentarios, luego los literales de cadena y carácter, después los números, seguidos de palabras reservadas e identificadores, operadores, separadores y finalmente los espacios en blanco. Este orden es importante porque evita ambigüedades y mejora la precisión del análisis.

En general, estos cambios hicieron que el analizador fuera más robusto, capaz de manejar errores y de procesar correctamente distintos elementos del lenguaje sin interrumpir su ejecución.

Extensiones y mejoras adicionales

Además de los casos vistos en la práctica, se realizaron algunas mejoras adicionales al analizador léxico para hacerlo más completo y cercano a un uso real.

Una de estas mejoras fue el soporte para anotaciones de Java, como @Override. Inicialmente, el carácter @ era detectado como un error léxico y la palabra Override como un identificador. Para solucionar esto, se agregó un nuevo tipo de token llamado ANNOTATION junto con su expresión regular correspondiente. También se consideró el orden dentro del patrón general, colocando este nuevo token antes del identificador para asegurar su correcta detección.

Otra mejora importante fue la lectura de archivos .java. En lugar de trabajar con un texto fijo dentro del código, se implementó el uso de Files.readString, lo que permite analizar directamente archivos reales. Esto hace que el programa sea más práctico y reutilizable.

También se agregó la funcionalidad de exportar los resultados a un archivo .txt. En lugar de mostrar los tokens únicamente en consola, ahora se generan líneas de texto que posteriormente se escriben en un archivo utilizando Files.write. Esto facilita la revisión de resultados y permite guardar evidencia de las pruebas realizadas.

Adicionalmente, se incorporó el manejo de columnas en los tokens. Para ello, se calculó la posición de cada token en función de su índice dentro del texto y la última aparición de un salto de línea. Esto permitió mostrar tanto la línea como la columna, mejorando la precisión de la información generada por el analizador.

2. Resultados de pruebas

2.1 Caso 1

2.1.1 Resultado Esperado

```
[L1 ] KEYWORD        → "public"
[L1 ] KEYWORD        → "class"
[L1 ] IDENTIFIER     → "MiClase"
[L2 ] KEYWORD        → "private"
[L2 ] KEYWORD        → "int"
[L2 ] IDENTIFIER     → "contador"
[L3 ] KEYWORD        → "public"
[L3 ] KEYWORD        → "void"
[L3 ] IDENTIFIER     → "incrementar"
```

2.1.2 Resultado Obtenido

```
[L1  ] KEYWORD        --> "public"
[L1  ] KEYWORD        --> "class"
[L1  ] IDENTIFIER     --> "MiClase"
[L1  ] UNKNOWN        --> "{"
[L2  ] KEYWORD        --> "private"
[L2  ] KEYWORD        --> "int"
[L2  ] IDENTIFIER     --> "contador"
[L2  ] UNKNOWN        --> ";"
[L3  ] KEYWORD        --> "public"
[L3  ] KEYWORD        --> "void"
[L3  ] IDENTIFIER     --> "incrementar"
[L3  ] UNKNOWN        --> "()"
[L3  ] UNKNOWN        --> "{"
[L4  ] IDENTIFIER     --> "contador"
[L4  ] UNKNOWN        --> "++;"
[L5  ] UNKNOWN        --> "}"
[L6  ] UNKNOWN        --> "}"
```

2.2 Caso 2

2.2.1 Resultado Esperado

```
IDENTIFIER  → "a"
OPERATOR    → "=="    ← NO "=" + "="
IDENTIFIER  → "b"
OPERATOR    → "&&"    ← operador compuesto
IDENTIFIER  → "c"
OPERATOR    → "!="
```

```
IDENTIFIER  → "d"
OPERATOR    → "||"
IDENTIFIER  → "i"
OPERATOR    → "++"    ← maximal munch: ++ no es + +
OPERATOR    → "<="
INTEGER_LIT → "10"
```

2.2.2 Resultado Obtenido

```
[L1  ] IDENTIFIER     --> "a"
[L1  ] OPERATOR       --> "=="
[L1  ] IDENTIFIER     --> "b"
[L1  ] OPERATOR       --> "&&"
[L1  ] IDENTIFIER     --> "c"
[L1  ] OPERATOR       --> "!="
[L1  ] IDENTIFIER     --> "d"
[L1  ] OPERATOR       --> "||"
[L1  ] IDENTIFIER     --> "i"
[L1  ] OPERATOR       --> "++"
[L1  ] OPERATOR       --> "<="
[L1  ] UNKNOWN        --> "10"
```

2.3 Caso 3

2.3.1 Resultado Esperado

```
INTEGERLIT      → "255"
HEXADECIMALLIT  → "0xFF"
BINARYLIT       → "0b1111_0000"
FLOATLIT        → "3.14159"
STRINGLIT       → "\"hola\\n\""
```

2.3.2 Resultado Obtenido

```
[L1  ] KEYWORD        --> "int"
[L1  ] IDENTIFIER     --> "a"
[L1  ] OPERATOR       --> "="
[L1  ] INTEGERLIT     --> "255"
[L1  ] SEPARATOR      --> ";"
[L2  ] KEYWORD        --> "int"
[L2  ] IDENTIFIER     --> "b"
[L2  ] OPERATOR       --> "="
[L2  ] HEXADECIMALLIT --> "0xFF"
[L2  ] SEPARATOR      --> ";"
[L3  ] KEYWORD        --> "int"
[L3  ] IDENTIFIER     --> "c"
```

```
[L3  ] OPERATOR       --> "="
[L3  ] BINARYLIT      --> "0b1111_0000"
[L3  ] SEPARATOR      --> ";"
[L4  ] KEYWORD        --> "double"
[L4  ] IDENTIFIER     --> "pi"
[L4  ] OPERATOR       --> "="
[L4  ] INTEGERLIT     --> "3"
[L4  ] SEPARATOR      --> "."
[L4  ] INTEGERLIT     --> "14159"
[L4  ] SEPARATOR      --> ";"
[L5  ] IDENTIFIER     --> "String"
[L5  ] IDENTIFIER     --> "s"
[L5  ] OPERATOR       --> "="
[L5  ] STRINGLIT      --> ""hola\n""
[L5  ] SEPARATOR      --> ";"
```

2.4 Caso 4

2.4.1 Resultado Esperado

```
[L2 ] JAVADOC      → "/** JavaDoc comment */"
[L3 ] KEYWORD      → "public"
[L3 ] KEYWORD      → "class"
[L3 ] IDENTIFIER   → "Prueba"
[L4 ] KEYWORD      → "public"
[L4 ] KEYWORD      → "static"
[L4 ] KEYWORD      → "void"
[L4 ] IDENTIFIER   → "main"
[L5 ] HEX_LIT      → "0xFF"
[L6 ] FLOAT_LIT    → "3.14"
[L7 ] STRING_LIT   → "\"mundo\""
[L8 ] OPERATOR     → "=="
[L8 ] OPERATOR     → "&&"
[L8 ] OPERATOR     → ">"
[L8 ] OPERATOR     → "++"
Error léxico [L9, C9]: '@' (U+0040)
```

2.4.2 Resultado Obtenido

```
Error léxico [L8, C9]: '@'
[L2  ] KEYWORD        --> "public"
[L2  ] KEYWORD        --> "class"
[L2  ] IDENTIFIER     --> "Prueba"
[L2  ] SEPARATOR      --> "{"
[L3  ] KEYWORD        --> "public"
[L3  ] KEYWORD        --> "static"
[L3  ] KEYWORD        --> "void"
[L3  ] IDENTIFIER     --> "main"
[L3  ] SEPARATOR      --> "("
```

```
[L3  ] IDENTIFIER     --> "String"
[L3  ] SEPARATOR      --> "["
[L3  ] SEPARATOR      --> "]"
[L3  ] IDENTIFIER     --> "args"
[L3  ] SEPARATOR      --> ")"
[L3  ] SEPARATOR      --> "{"
[L4  ] KEYWORD        --> "int"
[L4  ] IDENTIFIER     --> "x"
[L4  ] OPERATOR       --> "="
[L4  ] HEXADECIMALLIT --> "0xFF"
[L4  ] SEPARATOR      --> ";"
[L5  ] KEYWORD        --> "double"
[L5  ] IDENTIFIER     --> "pi"
[L5  ] OPERATOR       --> "="
[L5  ] FLOATLIT       --> "3.14"
[L5  ] SEPARATOR      --> ";"
[L6  ] IDENTIFIER     --> "String"
[L6  ] IDENTIFIER     --> "s"
[L6  ] OPERATOR       --> "="
[L6  ] STRINGLIT      --> ""mundo""
[L6  ] SEPARATOR      --> ";"
[L7  ] KEYWORD        --> "if"
[L7  ] SEPARATOR      --> "("
[L7  ] IDENTIFIER     --> "x"
[L7  ] OPERATOR       --> "=="
[L7  ] INTEGERLIT     --> "255"
[L7  ] OPERATOR       --> "&&"
[L7  ] IDENTIFIER     --> "pi"
[L7  ] OPERATOR       --> ">"
[L7  ] FLOATLIT       --> "0.0"
[L7  ] SEPARATOR      --> ")"
[L7  ] SEPARATOR      --> "{"
[L7  ] IDENTIFIER     --> "x"
[L7  ] OPERATOR       --> "++"
[L7  ] SEPARATOR      --> ";"
[L7  ] SEPARATOR      --> "}"
[L9  ] SEPARATOR      --> "}"
[L10 ] SEPARATOR      --> "}"
```

2.5 Resultados finales

Como parte de la validación del analizador léxico, se utilizaron archivos de prueba en formato .java, los cuales fueron procesados correctamente por el programa. A partir de estos, se generó un archivo de salida en formato .txt que contiene la lista de tokens identificados, junto con su tipo, línea y columna correspondiente.

Con el fin de facilitar la revisión de los resultados, a continuación se incluyen los enlaces tanto del archivo de entrada como del archivo de salida generado:

- Archivo de entrada (.java): Archivo de entrada - Ejemplo.java
- Archivo de salida (.txt): Archivo de salida - salida.txt

3. Preguntas de Discusión

3.1 Caso 1

1. **¿Qué pasa si eliminas los \b del patrón KEYWORD?**
   Si se eliminan los \b (límites de palabra), el analizador podría reconocer palabras reservadas dentro de identificadores. Por ejemplo, en contador, podría detectar int como keyword, lo cual es incorrecto. Los \b evitan ese problema asegurando coincidencias completas.
2. **¿Por qué el patrón KEYWORD debe evaluarse antes que IDENTIFIER?**
   Porque las palabras reservadas también cumplen con la forma de un identificador. Si primero se evalúa IDENTIFIER, entonces palabras como int o while serían clasificadas como identificadores en lugar de keywords.
3. **¿Podrías construir una sola ER que reconozca ambos tokens a la vez?**
   Sí, se podría hacer una sola expresión regular que reconozca identificadores y luego, ya en código, verificar si el lexema pertenece a la lista de keywords.

3.2 Caso 2

1. **¿Cómo manejarías el operador ternario ?: en tu analizador?**
   El operador ternario realmente se compone de dos operadores: ? y :. Por lo tanto, se pueden reconocer por separado dentro del patrón de operadores sin necesidad de tratarlos como uno solo.
2. **El operador >>>debe reconocerse antes que >>. ¿Por qué?**
   Porque si primero se reconoce >>, entonces al analizar >>>se dividiría incorrectamente como >>y luego >. Por eso, los operadores más largos deben ir primero en la expresión regular.
3. **¿Qué diferencia hay entre un separador y un operador desde la perspectiva de la gramática?**
   Los separadores (como ;, {, }) sirven para estructurar el código, mientras que los operadores (como +, =, ==) realizan operaciones o evaluaciones. Es decir, uno organiza la sintaxis y el otro define acciones.

3.3 Caso 3

1. **¿Por qué 08 NO es un octal válido en Java? ¿Cómo ajustarías la ER?**
   EPorque los números octales solo permiten dígitos del 0 al 7. El 8 no es válido en ese sistema. La expresión regular ya lo controla usando [0-7]+.
2. **¿Cómo extenderías el patrón STRING para soportar Text Blocks (”””...”””)?**
   Se tendría que agregar un nuevo patrón que reconozca triple comillas, algo como: `"""[\\s\\S]*?"""` y colocarlo antes del patrón de string normal para evitar conflictos.
3. **¿Tiene sentido que el analizador léxico verifique si un número está en rango?**
   No mucho. El lexer solo se encarga de reconocer la forma del token, no su valor. Validar rangos (como que un int no exceda su límite) es más responsabilidad del análisis semántico.

3.4 Caso 4

1. **¿Cómo manejaría tu lexer un comentario de bloque no cerrado al final del archivo?**
   Lo detectaría como un error léxico, ya que no se encuentra el cierre */. Se podría lanzar una excepción o registrar el error indicando que el comentario no fue cerrado correctamente.

2. **¿Deberían los JavaDoc comments producir un token diferente a los block comments?**
   Depende del objetivo. Si solo se quiere analizar sintaxis, podrían tratarse igual. Pero si se quiere extraer documentación, entonces sí conviene manejarlos como un token distinto (JAVADOC).

3. **¿Cuál es la diferencia entre recuperación de errores en el lexer vs en el parser?**
   En el lexer, la recuperación de errores consiste en ignorar o saltar caracteres inválidos para continuar el análisis. En el parser, implica intentar reconstruir la estructura del programa para seguir interpretándolo, lo cual es más complejo.